arXiv:2605.27452v1 [cs.CV] 24 May 2026

Fine-Tuning Vision-Language Models for Understanding Current Damage and Scoring Priority with Quality Guard Agent

Takato Yasuno

Abstract

Bridge inspection in Japan requires mandatory visual assessments every five years, yet qualitative damage ratings (levels a–e) assigned by different engineers exhibit significant inter-rater variability—a critical barrier to consistent infrastructure management. The aging of skilled engineers further threatens inspection capacity. This paper presents a methodology for automating bridge damage understanding and repair priority scoring using fine-tuned Vision-Language Models (VLMs).

We fine-tune LLaVA-1.5-7B with QLoRA (4-bit NF4 quantization, rank=32) on up to 4,000 paired bridge damage images and inspection text records, then evaluate on a fixed test set of 800 images. The model outputs natural language descriptions identifying structural members and damage patterns, from which a rule-based scoring engine calculates a five-level repair priority index. A progressive training study (1k/2k/3k/4k samples) reveals that 2k training samples achieve near-optimal validation loss (3.073) in only 2.9 hours of training; beyond 2k, validation loss improves by no more than 0.2% per doubling of training samples, exhibiting clear diminishing returns. Furthermore, semantic similarity on the held-out test set peaks at 3k (0.6909) and degrades at 4k (0.6739), indicating that quality-curated mid-scale data outperforms larger but noisier corpora. Inference optimization combining `torch.compile()` and batch processing (batch_size=8) achieves 10.06 seconds per image—a 70.2% reduction over the unoptimized baseline.

Our approach contributes to data governance in bridge inspection, reduces inter-rater variability, and provides AI-assisted triage to augment expert engineers in inspection workflows. Furthermore, we introduce a two-stage **Quality Guard** using a fine-tuned Swallow-8B SLM to reject low-quality VLM outputs before priority scoring, preventing spurious scores from damaged or unrecognised images.

Keywords: Vision-Language Models, Fine-Tuning, QLoRA, Bridge Inspection, Damage Assessment, Repair Priority Scoring, Infrastructure AI.

1 Introduction

Japan's road network encompasses approximately 730,000 highway bridges; nearly half of these will exceed 50 years of service life by 2033 [1]. Under the Road Act (amended 2014), every road bridge must undergo close-up visual inspection every five years, producing paired records: a photographic image of the damage site and a hand-written inspection note (*shoken*) describing the structural member, damage type, severity, and recommended corrective action.

Despite this systematic framework, two persistent challenges impede inspection quality. **First, inter-rater variability:** damage ratings (levels a–e) assigned by different engineers diverge due to subjective judgment, undermining data consistency across the national bridge stock [1]. **Second, workforce contraction:** experienced bridge inspectors are retiring faster than they can be replaced, threatening capacity at the required cadence.

Recent Vision-Language Models (VLMs) present a promising path forward. Models such as LLaVA [2], InstructBLIP [3], and GPT-4V [4] can generate detailed descriptive text from images; however, these general-purpose models have no exposure to specialised Japanese inspection vocabulary and cannot produce the structured output required by inspection standards.

This paper presents a methodology for **bridge damage understanding and repair priority scoring** via domain-adapted VLMs. We fine-tune LLaVA-1.5-7B [2] with QLoRA [5] on up to 4,000 paired bridge inspection records. We then deploy a rule-based scoring engine that converts the model's free-form descriptions into a five-level repair priority index (Immediate to Minimal).

Terminology and scope. We distinguish *damage understanding* from *damage description*. *Damage understanding* refers to the cognitive process by which an engineer interprets the condition of a structure along four complementary aspects: (1) the **current state** of surface damage visible in an image, (2) the **temporal progression** of deterioration across multiple inspection time points, (3) the **latent causal**

factors (environmental exposure, traffic load, material aging) inferred from domain knowledge and experience, and (4) the **heterogeneity of deterioration rates**—identifying accelerating versus stable clusters of structures or members. *Damage description*, by contrast, denotes the natural-language text produced as VLM inference output that records observable damage signals from an input image. This paper scopes the VLM-based methodology to aspect (1) only: damage understanding from a single current-time image, realised as a damage description that feeds the priority-scoring rule engine. Aspects (2)–(4) require longitudinal data, contextual metadata, and structure-level aggregation, and are positioned as future extensions in Section 5.8.

Contributions:

1. **Damage VLM**: the first QLoRA fine-tuning study of a 7B-parameter VLM on Japanese bridge inspection image–text pairs, achieving the *Acceptable* quality tier on a consumer GPU.

2. **Repair Priority Scoring**: a transparent, auditable rule engine that converts VLM free-form output into a five-level priority index, enabling consistent maintenance triage without subjective judgment.

3. **Quality Guard Agent**: a two-stage guard (rule-based Stage 1 + Swallow-8B SLM judge Stage 2) that rejects low-quality outputs before scoring, with thresholds calibrated from the empirical 5th/95th token-count percentiles of $n = 800$ predictions (12.5% low-quality rate identified).

4. **Visual Inspection ScoreBot**: an end-to-end local pipeline combining the Damage VLM with the Quality Guard and scoring engine—deployable without cloud API dependencies on a single consumer GPU.

This work builds on the authors' prior study of quantized LLaVA-1.5-7B for bridge damage assessment [6], which benchmarked INT4/INT8/BF16 quantization levels without task-specific fine-tuning. The multi-stage inspection framework in [7] further motivated the end-to-end pipeline presented here.

Section 2 reviews related work, including Section 2.5 on AI agent and quality guardrail methods. Section 3 details the system methodology. Section 4 presents evaluation results. Section 5 discusses findings and limitations. Section 6 concludes.

2 Related Work

2.1 Vision-Language Models for Structural Damage Understanding

The development of large VLMs accelerated with CLIP [8], which established scalable joint image–text representation via contrastive pre-training on 400 million web image–caption pairs. Flamingo [9] extended this to few-shot visual reasoning by conditioning frozen language models on visual features through gated cross-attention layers. BLIP-2 [10] and InstructBLIP [3] improved instruction-following by bridging frozen image encoders and language models via a lightweight querying transformer. LLaVA [2] simplified the paradigm by projecting visual tokens directly into a language model embedding space, enabling visual instruction tuning at reduced cost. Its successor LLaVA-1.5 [11], a CVPR 2024 highlight paper, showed that an MLP connector with CLIP-ViT-L-336 px achieves state-of-the-art across 11 benchmarks using only 1.2 M publicly available training samples — the same base model adopted in this work. MiniGPT-4 [12] demonstrated that aligning a frozen visual encoder with a large language model through a single projection layer suffices for rich image-description generation.

Foundational perception models have further expanded the toolkit for inspection. Segment Anything (SAM) [13] introduced a promptable model trained on over one billion masks that transfers zero-shot across diverse visual domains. Grounding DINO [14] combines transformer-based detection with language grounding for open-set localisation, complementary to the language-generation focus of generative VLMs.

In civil infrastructure, vision-based inspection has followed an independent trajectory focused on pixel-level detection. Cha et al. [15] demonstrated that convolutional neural networks detect surface cracks with near-expert accuracy. Spencer et al. [16] surveyed computer vision advances for structural inspection, noting the gap between pixel-level detection and the natural language reporting required in engineering practice. Dorafshan et al. [17] benchmarked CNN detectors for concrete crack localisation, and Hsieh and Tsai [18] compared machine learning approaches for crack detection.

Our work bridges these two streams: task-specific QLoRA fine-tuning of LLaVA-1.5 generates the structured natural language output required by Japanese inspection standards, going beyond pixel-level detection to holistic damage description in domain-appropriate Japanese vocabulary. Early work on few-

shot damage vision mining [19] showed that imbalanced data and rare damage events are fundamental challenges in infrastructure inspection datasets. A multi-stage bridge inspection system [7] integrated foundation models with location anonymisation for privacy-preserving field imaging. Our quantization study [6] benchmarked inference speed and description quality across LLaVA-1.5-7B precision levels, establishing INT4 QLoRA as the optimal speed–quality trade-off and motivating the domain fine-tuning step presented in this paper.

2.2 AI-Assisted Decision Support for Infrastructure Maintenance

Bridge management has long relied on formal decision-support frameworks. Frangopol [20] formalised life-cycle performance optimisation for structural systems under uncertainty, establishing the theoretical basis for risk-based inspection prioritisation. Operational bridge management systems encode engineering heuristics into rule-based priority scores but depend on consistent structured input that manual inspection cannot always guarantee.

Yang et al. [21] applied deep CNNs to automatically map surface cracks on dam structures, reducing manual effort in routine assessments. Such detection-focused systems, however, lack the semantic understanding needed to recommend maintenance actions or to assign priority across heterogeneous damage types.

More recently, vision-language models have been adapted for zero-shot industrial inspection. WinCLIP [22] (CVPR 2023) introduced a compositional ensemble on window/patch/image-level CLIP features, achieving 91.8% AUROC in zero-shot anomaly classification on MVTec-AD without task-specific training. AnomalyCLIP [23] (ICLR 2024) extended this line by learning object-agnostic text prompts that capture generic normality and abnormality, enabling strong zero-shot transfer across 17 datasets spanning defect inspection and medical imaging. AnomalyGPT [24] (AAAI 2024) demonstrated that large vision-language models fine-tuned with simulated anomalous images and corresponding textual descriptions achieve state-of-the-art industrial anomaly detection without manual threshold setting, reaching 94.1% image-level AUC with only one normal reference shot.

Our scoring engine addresses the bridge-specific gap: rather than detecting anomalies from scratch, it consumes VLM-generated Japanese damage descriptions and applies YAML-defined weights for member type, severity, location, and extent to produce a five-level repair priority index. Domain experts can audit and update the scoring logic without machine learning expertise, preserving transparency for safety-critical decisions. Complementary work on heterogeneous graph importance scoring for bridge networks [25] demonstrated that LLM-generated interpretations effectively bridge algorithmic rankings and maintenance decision-makers, reinforcing the value of language as an interface between AI outputs and engineering judgment.

2.3 Multimodal Systems for Industrial Visual Inspection

The emergence of GPT-4V [4] catalysed interest in general-purpose multimodal systems for industrial applications. Cao et al. [26] systematically evaluated GPT-4V on anomaly detection across four modalities and nine tasks, demonstrating promising zero-shot and one-shot performance while highlighting limitations in fine-grained localisation of subtle defects. Prompt-engineered GPT-4V can identify manufacturing defects and answer technical questions from product photographs, yet proprietary API dependence, latency, and data privacy constraints limit deployment in regulated infrastructure contexts.

Open-source VLMs fine-tuned or adapted for domain-specific inspection offer a more practical path. Anomaly-OV [27] (CVPR 2025) proposed a specialist visual assistant for zero-shot anomaly detection and reasoning, introducing a Look-Twice Feature Matching mechanism that adaptively emphasises abnormal visual tokens, achieving significant improvements over generalist MLLMs on industrial and medical benchmarks. Zhang et al. [28] (CVPR 2025) demonstrated training-free anomaly detection using vision and language foundation models, eliminating fine-tuning while maintaining competitive performance on standard benchmarks — contrasting with our QLoRA-based domain adaptation strategy. LogicAD [29] (AAAI 2025) combined VLM-based text feature extraction with logical reasoning to produce explainable anomaly reports, aligning with the engineering interpretability requirements of infrastructure operators.

Our **Visual Inspection ScoreBot** operationalises this paradigm for Japanese bridge infrastructure: the Damage VLM generates free-form Japanese damage descriptions; a regex-based extractor converts them to structured JSON attributes; and the scoring engine produces actionable priority scores — all running locally on a single consumer GPU without external API calls. This architecture aligns with emerging industrial AI patterns in which a language model serves

as the semantic front-end to a deterministic rule engine [2, 5], combining interpretability with the expressive power of neural language understanding.

2.4 Automation and Robotics in the Construction Domain

The International Symposium on Automation and Robotics in Construction (ISARC) provides a focused forum for construction-domain AI, and contributions from ISARC 2023–2025 reveal three convergent themes directly relevant to this paper: (i) language-model-driven extraction from inspection documents, (ii) multimodal report generation from visual data, and (iii) deep-learning-based structural damage detection coupled with maintenance scheduling.

Language models for inspection document processing. Omar and Moselhi [30] (ISARC 2023) established an integrated pipeline for automated acquisition and parsing of bridge inspection reports, demonstrating that text-mining can replace manual data entry for deck-deficiency records. Their follow-on study [31] (ISARC 2025) replaced rule-based parsing with fine-tuned Generative Pre-trained Transformers, achieving substantial accuracy gains in recognising concrete-defect entities from free-text engineering records — a direct antecedent of the QLoRA-based Damage VLM strategy presented here, which adapts the same fine-tuning paradigm to Japanese inspection vocabulary. Reja et al. [32] (ISARC 2025) compared few-shot LLM classifiers against fine-tuned transformers for road maintenance log classification, confirming that domain-adapted language models consistently outperform general-purpose prompting in specialised infrastructure contexts.

Multimodal and vision-language approaches. Pu et al. [33] (ISARC 2024) employed multimodal large language models with Set-of-Mark prompting to generate construction inspection reports directly from annotated site photographs — the closest prior work to our Damage VLM pipeline, though targeting general construction quality in English rather than bridge-specific damage in Japanese regulatory vocabulary. Hsu et al. [34] (ISARC 2024) released VL-Con, a construction-domain vision-language dataset, reinforcing the finding that task-specific data collection is a prerequisite for reliable VLM performance in AEC applications and motivating the curated Japanese instruction dataset described in Section 3.2. Mengiste et al. [35] (ISARC 2025) automated weekly construction progress reporting using multimodal LLM workflows, further illustrating the industrial demand for language-based interfaces to visual inspection data.

Structural damage detection and maintenance decision support. Assad et al. [36] (ISARC 2025) benchmarked single-stage versus two-stage detectors for bridge crack detection and segmentation, confirming that pixel-level localisation methods remain the baseline for damage mapping, though without generating the natural-language descriptions required for regulatory reporting. Zuo et al. [37] (ISARC 2025) combined transformer-based detection with multi-resolution 3D reconstruction to spatially localise bridge surface defects, demonstrating the complementary role of geometric modelling alongside 2D image-based language generation. Núñez Varillas et al. [38] (ISARC 2024) integrated UAV photogrammetry with CNN-based concrete damage classification on reconstructed bridge 3D models, illustrating how remote sensing reduces inspector exposure while maintaining detection accuracy. At the maintenance-decision level, Alsharqawi et al. [39] (ISARC 2025) proposed an automated scheduling tool for bridge maintenance, repair, and replacement using multi-objective genetic algorithms. Their portfolio-level optimiser is complementary to our inspection-level priority scoring engine: the five-level repair priority index produced by the ScoreBot provides the per-element damage state that such optimisers require as input.

Collectively, these ISARC contributions confirm two transitions in construction inspection practice: automated language processing is displacing manual report writing, and multimodal architectures are superseding single-modality vision pipelines. The present work advances both transitions by fine-tuning a state-of-the-art open-source VLM (LLaVA-1.5-7B) on a curated Japanese bridge inspection dataset, delivering domain-appropriate structured damage descriptions without dependence on proprietary APIs or manual post-processing.

2.5 AI Agent and Quality Guardrail

The reliability of VLM-generated descriptions in safety-critical domains critically depends on robust output quality control. Three complementary lines of research are directly relevant to the Quality Guard introduced in this work.

LLM-as-Judge and quality evaluation. Zheng et al. [40] introduced the LLM-as-a-Judge paradigm through MT-Bench and Chatbot Arena, demonstrating that a strong language model can serve as an effective automated quality evaluator for free-text outputs. Building on this, Liu et al. [41] proposed G-Eval, a framework that employs GPT-4 with chain-of-thought to evaluate natural language generation qual-

ity with higher human alignment than conventional n-gram metrics. Min et al. [42] introduced FActScore to measure the factual precision of long-form text by decomposing outputs into atomic facts and verifying each independently. These judge-based evaluation frameworks motivate using a fine-tuned SLM (Swallow-8B [43]) as a domain-aware quality evaluator for VLM damage descriptions, following the principle of "LLM judging LLM" without relying on proprietary external APIs.

Guardrails and self-correction in LLM pipelines. Constitutional AI [44] proposed training AI systems to critique and revise their own outputs according to a set of principles, establishing the self-correction paradigm that underlies quality guardrail architectures. NeMo Guardrails [45] formalised this as a programmable rail system, allowing developers to define topical, safety, and format constraints that intercept LLM outputs before they reach downstream applications — directly analogous to the quality gate in our v0.6 pipeline. Self-RAG [46] extended the concept to retrieval-augmented generation, where a model learns to reflect on whether retrieval and generation steps should be invoked, achieving selective output through special reflection tokens. Chain-of-Verification [47] reduced hallucination in LLM outputs by generating independent verification questions and cross-checking initial responses against them. These approaches collectively inform our two-stage guard design: Stage 1 applies deterministic rules (token-count thresholds and repetition detection) analogous to structural rails; Stage 2 invokes Swallow-8B as an LLM judge for borderline cases, combining speed with semantic depth.

AI agents with self-reflection. ReAct [48] combined reasoning traces with action steps in language agents, enabling dynamic adjustment of the inference trajectory based on observed feedback. Reflexion [49] built on this by introducing verbal reinforcement signals from a linguistic reflection mechanism, allowing agents to revise strategies across trials without gradient updates. Shen et al. [50] surveyed alignment techniques for large language models, identifying output quality guardrails as a key mechanism for deploying LLMs in high-stakes applications.

In the bridge inspection domain, the consequence of false-high quality scores is misallocated maintenance budgets and deferred safety-critical repairs. By interposing an SLM judge between the VLM and the rule-based priority scoring engine, the Quality Guard addresses the domain-specific risk that VLMs trained on limited data may produce plausible but content-sparse descriptions for images that are overexposed, occluded, or low-resolution — cases where assigning a priority score would be misleading rather than informative. Our guard distinguishes two failure modes: *"Not recognised from only image"* (insufficient visual information for any damage description) and *"Dirty or Noisy image"* (repetitive or incoherent output). Only descriptions passing both stages receive a five-level repair priority score; failed samples return *"No Score due to Low Quality Image"*, preserving the integrity of the priority ranking for downstream maintenance planning.

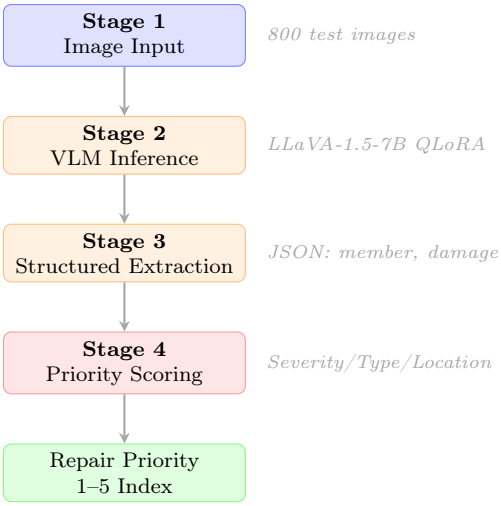


Figure 1: End-to-end pipeline for bridge damage understanding and repair priority scoring. Fine-tuning (Section 3.3) targets Stage 2; Stages 3–4 remain rule-based.

3 Methodology

3.1 System Architecture Overview

Figure 1 illustrates the end-to-end pipeline comprising four sequential stages: image input, VLM-based damage understanding, structured information extraction, and repair priority scoring. A detailed algorithm flow that incorporates the Quality Guard Agent is shown in Figure 11 (Section 3.7).

3.2 Dataset Construction

3.2.1 Source Data

We collect 10,789 paired bridge damage images and Japanese inspection text records from routine close-up visual inspections conducted under Japan's Road Act, which mandates five-year periodic bridge inspections. Each record contains a damage image and the corresponding inspector's *shooken* (observation text,

Japanese: *shoken*) written using standardised civil engineering terminology.

3.2.2 Quality Filtering

To ensure training data quality, we apply the following filters:

- **Text length**: 15–500 characters (removes empty and excessively verbose records).
- **Invalid patterns**: excludes records containing only placeholder text ("nashi" (none), "???", or similar).
- **Keyword requirement**: at least one structural member term (e.g., *girder, deck, pier*) and one damage term (e.g., *rebar exposure, crack, corrosion*) must be present.

3.2.3 Stratified Sampling and Split Strategy

After filtering, we construct training and evaluation sets using stratified sampling by cross-product of member type and damage type to ensure class balance.

- **Fixed test set**: 800 samples drawn with random seed 42, held out for all experiments (798 with valid images, 2 missing).
- **Progressive training sets**: independently sampled sets of 1k, 2k, 3k, and 4k images to study data-scaling behaviour. Each training set uses an 80/20 train/validation split. This progression follows the scale-search methodology in [51], which identified $n \approx 4{,}000$ as optimal for Japanese domain-specific language models—extended here to the multimodal VLM setting.

Table 1 shows the resulting dataset statistics.

Table 1: Dataset statistics after quality filtering.

Split	Images	Valid (%)
Test (fixed)	800	798 (99.75%)
Train-1k (train/val)	799 / 200	100%
Train-2k (train/val)	1,599 / 400	100%
Train-3k (train/val)	2,398 / 600	100%
Train-4k (train/val)	3,196 / 799	100%
Source pool	10,789	—

3.3 QLoRA Fine-Tuning

3.3.1 Base Model

We fine-tune **LLaVA-1.5-7B** [2] (HuggingFace identifier: `llava-hf/llava-1.5-7b-hf`), an instruction-tuned vision-language model combining CLIP ViT-L/14 as the vision encoder and Vicuna-1.5-7B as the language decoder connected via a two-layer MLP projection. LLaVA-1.5-7B is chosen for its strong visual instruction-following capability and practical size for consumer GPU deployment.

3.3.2 QLoRA Configuration

We apply **QLoRA** [5] (Quantized Low-Rank Adaptation), which loads the base model in 4-bit NF4 precision while training low-rank adapter matrices in full precision. This enables fine-tuning a 7B-parameter model within 16GB VRAM.

The quantization configuration uses 4-bit NF4 with double quantization (compressing quantization constants themselves) via `bitsandbytes` [52]. Low-rank adapters are inserted into all major projection matrices as shown in Table 2.

The adapter scaling factor is $s = \alpha/r = 64/32 = 2$, controlling the magnitude of the learned residual updates relative to the frozen base weights. The LoRA decomposition for each weight matrix $W_0 \in \mathbb{R}^{d\times k}$ is:

$$W = W_0 + \frac{\alpha}{r} BA \tag{1}$$

where $B \in \mathbb{R}^{d\times r}$ and $A \in \mathbb{R}^{r\times k}$ are the low-rank adapter matrices initialised as $B = \mathbf{0}$ and $A \sim \mathcal{N}(0, \sigma^2)$.

3.3.3 Progressive Training Strategy

To investigate data-scaling behaviour, we train four independent models on progressively larger datasets (1k, 2k, 3k, 4k) using identical hyperparameters. Table 3 reports validation loss and training duration for each scale.

Three key observations emerge:

1. **1k → 2k**: Validation loss decreases by 1.98%, the largest single-step improvement, confirming benefit of additional data.
2. **Plateau at 2k–3k**: Loss stagnates at 3.073 despite 1.6× more training time, indicating convergence of the fine-tuning signal.
3. **Marginal gain at 4k**: Only 0.2% improvement over 3k at 1.4× cost, demonstrating strong diminishing returns.

Table 2: QLoRA and training hyperparameters.

Parameter	Value
Quantization	
Precision	4-bit NF4
Double quantization	Enabled
Compute dtype	float16
LoRA Adapters	
Rank r	32
Alpha α	64
Dropout	0.05
Target modules	`q,k,v,o_proj; gate,up,down_proj`
Training	
Batch size	4
Gradient accumulation	4 (eff. batch = 16)
Learning rate	2×10^{-4}
Epochs	3
Warmup steps	50
Max gradient norm	1.0
Weight decay	0.01
Optimizer	AdamW
LR scheduler	Cosine
Mixed precision	FP16
Hardware	
GPU	RTX 4060 Ti 16GB
CUDA	12.4
PyTorch	2.6.0+cu124
Transformers	4.57.6
PEFT	0.18.1

The **2k model** therefore offers the optimal cost–benefit trade-off: near-maximum accuracy ($\Delta = 0.2\%$ from 4k) achieved in 2.9 hours—2.2× faster than the 4k baseline.

3.4 Inference Optimisation

Running inference on 800 test images with an unoptimised HuggingFace `generate()` call requires 33.79 seconds per image (7.5 hours total), making systematic multi-model evaluation impractical. We implement three complementary optimisations.

3.4.1 `torch.compile()` Graph Optimisation

PyTorch 2.0 introduced `torch.compile()` [53], which applies TorchDynamo bytecode analysis and TorchInductor ahead-of-time compilation to the model's computation graph. We use `mode="reduce-overhead"`, which minimises Python interpreter overhead—the dominant bottleneck for repeated single-batch inference calls. An initial warm-up batch (≈60–80 seconds) is followed by accelerated subsequent inference.

Table 3: Progressive training results (LLaVA-1.5-7B QLoRA).

Data	Val Loss	Duration	Best Ckpt
1k	3.135	1:22:57	step-100/150
2k	3.073	2:55:37	step-300/300
3k	3.073	4:31:44	step-400/450
4k	3.067	6:19:10	step-600/600

3.4.2 Batch Processing

We implement a batched inference loop that processes B images simultaneously, padding inputs to uniform length within each batch:

$$\hat{T}_{\text{total}} = \underbrace{t_{\text{compile}}}_{\text{once}} + \left\lceil \frac{N}{B} \right\rceil \cdot t_{\text{batch}}(B) \tag{2}$$

where $N = 800$ is the test set size and $t_{\text{batch}}(B)$ is the per-batch latency at batch size B.

3.4.3 Batch Size Selection

We empirically evaluate batch sizes $B \in \{4, 8, 16\}$ on the RTX 4060 Ti (16GB VRAM). Table 4 reports per-image latency and GPU memory utilisation.

Table 4: Batch size optimisation results (2k model, 16 test samples).

Batch	s/image	VRAM	Status
Baseline (v0.4)	33.79	—	Reference
$B = 4$	16.80	<80%	✓
$B = 8$	10.06	≈78%	✓ **Optimal**
$B = 16$	—	98%	OOM risk

$B = 8$ reduces per-image latency by 70.2% relative to the baseline (33.79 → 10.06 seconds) while keeping GPU memory below the 80% safety threshold. $B = 16$ saturates VRAM at 15.7/16.0 GB, inducing long `torch.compile()` compilation and out-of-memory risk; it is therefore excluded.

3.4.4 Token Budget Optimisation

We analyse per-model output token distributions on 40-sample pilot runs to set `max_new_tokens` without systematic truncation:

Table 5: Output token statistics by training scale.

Model	Mean	Std	max new tokens
1k	348.5	148.4	512
2k	279.7	84.0	384
3k	266.4	160.0	384
4k	231.6	70.8	384

More training data produces more concise outputs (33.5% token reduction from 1k to 4k), an inverse-scaling phenomenon we attribute to improved instruction following. Setting `max_new_tokens`= 384 eliminates truncation for 2k and 4k models (10% and 0% truncation, respectively) while reducing generation time by 25% over the conservative default of 512.

We additionally apply `repetition_penalty`= 1.2 to suppress the looping behaviour observed in 1k model outputs.

3.4.5 Final Optimised Configuration

The combined optimisation (v0.5.1) achieves:

- **10.06 s/image** (batch_size=8, compiled)
- **2.2 hours** for 800 images per model
- **6.7 hours** for three models (2k/3k/4k) combined

compared to 22.5 hours with the unoptimised v0.4 script—a 70.2% wall-clock reduction enabling overnight evaluation of all four model scales.

3.5 Structured Information Extraction

The VLM outputs free-form Japanese text describing the damage scene. We extract five structured attributes via a rule-based parser:

$$j_i = \big(\underbrace{m_i}_{\text{member}} , \underbrace{d_i}_{\text{damage}} , \underbrace{l_i}_{\text{location}} , \underbrace{v_i}_{\text{severity}} , \underbrace{e_i}_{\text{extent}} \big) \quad (3)$$

Extraction uses a two-pass strategy: (1) keyword matching against a civil-engineering vocabulary dictionary to identify candidate spans, and (2) negation-aware pattern filtering to exclude "no damage" clauses. Table 6 defines each attribute and its vocabulary.

3.6 Repair Priority Scoring System

3.6.1 Score Computation

Given structured attributes j_i, the repair priority score $s_i \in [0, 1]$ is computed as a weighted sum:

Table 6: Structured extraction attributes.

Attribute	Example values
Member (m)	girder, deck, pier, bearing, railing
Damage (d)	rebar exposure, crack, corrosion, spalling, section loss
Location (l)	bottom face, upper surface, joint, support
Severity (v)	high, medium, low
Extent (e)	local, widespread, 30%, $1.5\,\mathrm{m}^2$

$$s_i = w_d\,\phi_d(d_i) + w_v\,\phi_v(v_i) + w_l\,\phi_l(l_i) + w_r\,\phi_r(r_i) \quad (4)$$

where $\phi_* : \text{category} \to [0, 1]$ are look-up functions defined in a YAML rule file, and the weights are:

$$(w_d,\, w_v,\, w_l,\, w_r) = (0.35,\ 0.40,\ 0.15,\ 0.10) \quad (5)$$

with $w_d + w_v + w_l + w_r = 1.0$. Severity carries the highest weight (40%) reflecting its direct relationship to structural safety; damage type is second (35%) because certain damage modes (rebar exposure, section loss) carry inherent structural risk regardless of assessed severity.

3.6.2 Damage-Type and Location Scores

Table 7 lists key look-up values derived from Japanese bridge inspection guidelines [1].

3.6.3 Combination Bonuses

Certain co-occurring attribute pairs trigger additive bonuses δ to capture compound structural risk that weighted summation alone may underestimate:

$$s_i' = \min\left(1.0,\ s_i + \sum_k \delta_k \cdot \mathbb{1}[\text{cond}_k(j_i)]\right) \quad (6)$$

For example: girder-bottom section loss ($\delta = +0.10$), high-severity rebar exposure ($\delta = +0.08$), and bearing corrosion ($\delta = +0.07$).

3.6.4 Five-Level Priority Index

The continuous score s_i' is discretised into a five-level repair priority index:

Table 7: Selected scoring rule look-up values.

Category	ϕ	Note
Damage type ϕ_d		
Section loss	1.00	Structural emergency
Rebar exposure	0.95	Corrosion propagation
Corrosion	0.85	Durability risk
Spalling	0.75	Cover loss
Crack	0.60	Context-dependent
Stain	0.30	Aesthetic only
Location ϕ_l		
Girder bottom face	1.00	Primary load path
Girder	0.95	
Bearing	0.90	Load transfer
Expansion joint	0.80	Water ingress
Deck	0.75	
Railing	0.40	Non-structural
Severity ϕ_v		
High	1.00	
Medium	0.60	
Low	0.30	

$$P_i = \begin{cases} 5 & s'_i \geq 0.85 \quad \text{(Immediate)} \\ 4 & s'_i \geq 0.70 \quad \text{(High)} \\ 3 & s'_i \geq 0.50 \quad \text{(Moderate)} \\ 2 & s'_i \geq 0.35 \quad \text{(Low)} \\ 1 & s'_i < 0.35 \quad \text{(Minimal)} \end{cases} \tag{7}$$

Priority 5 triggers immediate-repair alerts; Priority 1 records a finding without scheduling intervention.

3.7 Quality Guard Agent

To prevent low-quality VLM outputs from generating spurious repair priority scores, we interpose a two-stage **Quality Guard Agent** between the VLM inference and the scoring engine. Only descriptions that pass both stages receive a five-level priority score; failed samples return *"No Score due to Low Quality Image"*, preserving the integrity of the priority ranking for downstream maintenance planning. Figure 11 illustrates the complete algorithm flow of the Visual Inspection ScoreBot v0.6.3 pipeline.

3.7.1 Empirical Threshold Derivation

Rather than setting token-count thresholds from domain heuristics alone, we first analyse the distribution of output lengths over all $n = 800$ test-set predictions. Figure 2 presents (left) a violin plot of the token-count distribution and (right) the breakdown of low-quality patterns within the 5th and 95th percentile tails.

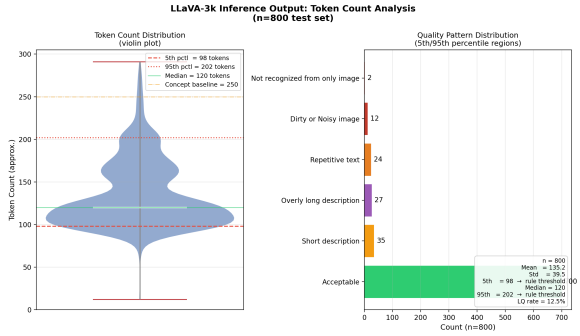


Figure 2: LLaVA-3k inference output token analysis ($n = 800$). *Left*: violin plot showing the token-count distribution (mean = 135.2, std = 39.5, median = 120). Red dashed/dotted lines mark the empirically derived rule thresholds ($\theta_{\text{low}} = 98$ at 5th pct; $\theta_{\text{high}} = 202$ at 95th pct). *Right*: quality pattern breakdown within the threshold regions. Acceptable outputs (87.5%) dominate; low-quality patterns total 12.5% across five failure categories.

The violin shape reveals a principal density mass between 100 and 220 tokens with a narrow waist below 100, indicating that genuinely informative damage descriptions cluster in a well-defined range. We set the token thresholds at empirical percentiles:

$$\theta_{\text{low}} = 98 \text{ (5th pct)}, \quad \theta_{\text{high}} = 202 \text{ (95th pct)} \tag{8}$$

This data-driven calibration avoids the twin risks of an over-tight lower bound (rejecting valid short descriptions) and an over-loose upper bound (admitting repetitive or hallucinated long outputs). Table 8 summarises the quality-pattern breakdown.

Table 8: Low-quality pattern breakdown ($n = 800$, 3k model).

Pattern	Count	%
Acceptable	700	87.5
Short description	35	4.4
Overly long description	27	3.4
Repetitive text	24	3.0
Dirty or Noisy image	12	1.5
Not recognised from only image	2	0.2
Low-quality total	100	12.5

3.7.2 Scope Definition

A critical design decision is the evaluation scope. As introduced in Section 1, this work targets aspect (1) of damage understanding—current-state observation from a single image—so the Quality Guard is calibrated against this scope rather than against multi-period or causal reasoning. Inspection text generated by VLMs trained on real inspection records tends to reproduce the reporting style of the training data, including references to previous inspection comparisons (*"since the last inspection..."*) and maintenance recommendations (*"corrective action is required"*). These out-of-scope clauses are artefacts of the training distribution, not hallucinations in the strict sense.

We therefore operationalise the scope as a *"one image, current time limited"* criterion: quality is assessed exclusively on the current-state damage observation. Out-of-scope content (two-time-point comparisons, maintenance directives) is **not penalised** provided that current damage information is present; only descriptions that consist *entirely* of out-of-scope content, with no damage information for the current image, are classified as FAIL. This design preserves useful damage signals while filtering outputs that carry no actionable inspection content. The broader implications of this scope choice, including extensions to aspects (2)–(4), are revisited in Section 5.8.

3.7.3 Stage 1: Rule-Based Filter

Stage 1 applies four deterministic rules to each prediction text $\hat{t}_i$ at negligible cost (≈0.01 s per sample, CPU-only):

1. **File existence** (Stage 0 pre-filter): if the source image is absent, return *"No such file or directory"* (FAIL).
2. **Token count**: if $|\hat{t}_i| < \theta_{\text{low}} = 98$, return *"Short description"* (FAIL).
3. **Repetition detection**: if a repeated n-gram pattern exceeds a coverage threshold, return *"Dirty or Noisy image"* (FAIL).
4. **Keyword absence**: if no structural damage keyword is present and vague, out-of-scope phrases dominate, return *"Not recognised from only image"* (FAIL).

Predictions passing all four rules proceed to Stage 2. Stage 1 alone captures 90.9% of outputs as *High Quality (rule only)* at a throughput of ≈0.01 s/row, enabling rapid triage without GPU resources.

3.7.4 Stage 2: SLM-as-Judge

Stage 2 invokes **Swallow-8B** [43] (`tokyotech-llm/Llama-3-Swallow-8B-Instruct-v0.1`), a Japanese instruction-tuned SLM, as a semantic quality judge following the LLM-as-a-Judge paradigm [40]. The model receives the extracted text with a structured prompt specifying the evaluation scope and returns a verdict in a fixed two-line format:

```
VERDICT: <PASS | FAIL>
REASON_CODE: <High Quality | Short description
             | Dirty or Noisy image
             | Not recognised from only image>
```

Inference runs under the **Unsloth** [54] framework (`FastLanguageModel.from_pretrained` + `for_inference()`) with 4-bit NF4 quantisation, fitting within the 16 GB VRAM budget after LLaVA-1.5-7B is unloaded. Batch inference at $B = 8$ (Section 3.4) achieves **10.10 seconds per guarded image** end-to-end on the RTX 4060 Ti.

The system prompt explicitly states the one-image/current-time scope and instructs the judge to disregard out-of-scope clauses when evaluating whether damage content is present.

3.7.5 Quality Guard Outcomes

Table 9 reports the final Quality Guard Agent outcomes over all $n = 800$ test samples from the v0.6.3 pipeline. All failures are captured at Stage 1 (rule-based filter); Stage 2 (Swallow-8B SLM judge) confirms all 727 Stage 1 PASS samples as *High Quality*, adding no additional rejections. This indicates that the empirically calibrated rule thresholds (Section 3.7.3) already cover the observable low-quality patterns in the 3k model's output distribution.

Table 9: Quality Guard Agent final outcomes ($n = 800$, 3k model, v0.6.3 pipeline). All FAILs originate at Stage 1; Stage 2 yields no additional rejections.

Outcome	Count	%
PASS — High Quality	727	90.9
FAIL — Dirty or Noisy image	36	4.5
FAIL — Short description	35	4.4
FAIL — No such file	2	0.2
FAIL total	73	9.1

3.8 Evaluation Protocol

3.8.1 Vector Similarity Metric

Rather than exact string matching, we evaluate generation quality using **cosine similarity in semantic embedding space**. Ground-truth inspection texts and model predictions are encoded with `sonoisa/sentence-bert-base-ja-mean-tokens-v2` [55], a Japanese Sentence-BERT model producing 768-dimensional sentence vectors pre-trained on Japanese Wikipedia and SNLI-translated corpora.

$$\text{sim}(t_i^{\text{gt}}, t_i^{\text{pred}}) = \frac{\mathbf{e}_i^{\text{gt}} \cdot \mathbf{e}_i^{\text{pred}}}{\|\mathbf{e}_i^{\text{gt}}\| \, \|\mathbf{e}_i^{\text{pred}}\|} \tag{9}$$

where $\mathbf{e} = \text{SentBERT}(t) \in \mathbb{R}^{768}$. We report mean cosine similarity $\bar{\rho}$ and its standard deviation σ_ρ over all 798 valid test pairs.

3.8.2 Quality Tier Classification

We map continuous similarity scores to five quality tiers to provide actionable assessments for model selection:

Table 10: Quality tier classification thresholds.

Tier	Threshold	Interpretation
Excellent	$\rho \geq 0.85$	Near-expert match
Good	$\rho \geq 0.75$	Minor omissions only
Acceptable	$\rho \geq 0.65$	Usable for triage
Poor	$\rho \geq 0.50$	Significant gaps
Very Poor	$\rho < 0.50$	Unreliable

A model achieving $\bar{\rho} \geq 0.65$ (Acceptable tier) is considered viable for production deployment as an AI-assisted triage tool.

3.8.3 Baseline Reference

The 1k model evaluated on the fixed 800-sample test set establishes the baseline: $\bar{\rho}_{1k} = 0.6491 \pm 0.0875$, placing it in the *Acceptable* tier. The 2k model achieves $\bar{\rho}_{2k} = 0.6850 \pm 0.0814$ (+5.5% relative improvement), remaining in the *Acceptable* tier but approaching the *Good* boundary (≥ 0.70). Comparative results for the 3k and 4k models will be added upon completion of inference.

3.9 Implementation and Reproducibility

Algorithm 1 summarises the core inference and evaluation loop in pseudocode, complementing the high-level pipeline view shown in Figure 11: the algorithm focuses on the per-sample VLM inference, Quality Guard invocation, and scoring steps, whereas the figure depicts the end-to-end v0.6.3 data flow including dataset loading and outcome aggregation.

Table 11: Cosine similarity by training scale (800 test samples). Tier thresholds: Excellent≥0.85, Good≥0.70, Acceptable≥0.60, Poor≥0.50.

Model	$\bar{\rho}$	σ	Tier	s/image
1k	0.6491	0.0875	Acceptable	33.79
2k	0.6850	0.0814	Acceptable	10.06
3k	0.6909	0.0784	Acceptable	7.50
4k	0.6739	0.0795	Acceptable	6.17

Algorithm 1 Inference and evaluation pipeline

Require: Model adapters θ^*, test set $\mathcal{D}_{\text{test}}$
1: Load base LLaVA-1.5-7B in 4-bit NF4 (`bitsandbytes`)
2: Merge QLoRA adapters θ^* via PEFT
3: Apply `torch.compile(mode=''reduce-overhead'')`
4: **for** each batch $\mathcal{B} \subseteq \mathcal{D}_{\text{test}}$, $|\mathcal{B}| = 8$ **do**
5: Tokenise and pad $\{(I_i, p)\}_{i \in \mathcal{B}}$
6: $\hat{T}_{\mathcal{B}} \leftarrow$ `model.generate(..., max_new_tokens=384)`
7: Decode $\hat{T}_{\mathcal{B}}$; append to results CSV
8: **end for**
9: **for** each sample i **do**
10: $\mathbf{e}_i^{\text{gt}} \leftarrow \text{SentBERT}(t_i^{\text{gt}})$
11: $\mathbf{e}_i^{\text{pred}} \leftarrow \text{SentBERT}(\hat{t}_i)$
12: $\rho_i \leftarrow \text{cosine}(\mathbf{e}_i^{\text{gt}}, \mathbf{e}_i^{\text{pred}})$
13: **end for**
14: Report $\bar{\rho}$, σ_ρ, and quality-tier distribution

All code, configuration files, and dataset preparation scripts are publicly available at `https://github.com/tk-yasuno/damage_vlm_finetune`. The repository includes:

- `train_v03_qlora.py` — fine-tuning script
- `inference_v051_qlora.py` — optimised inference
- `src/evaluation/vector_similarity_evaluator.py` — evaluation
- `models/scoring_rules.yaml` — priority scoring rules
- `config.yaml` — reproducible hyperparameter configuration

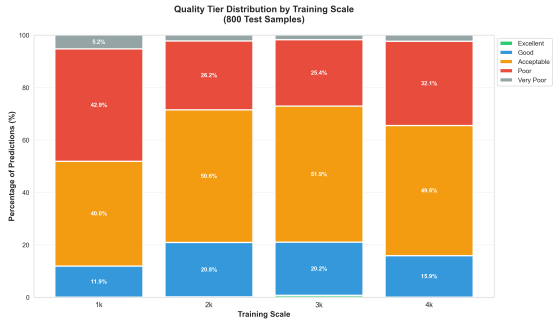


Figure 3: Quality tier distribution (stacked, 100% basis). The 2k model achieves 46.9% Good+ predictions, while 4k degrades to 15.9%, demonstrating the cost of over-expansion beyond optimal training scale.

4 Results

4.1 Semantic Similarity by Training Scale

Table 11 summarises cosine similarity for all four progressively trained models (1k–4k) evaluated on the fixed 800-sample test set. Moving from 1k to 2k training samples yields a gain of +0.0359 in mean cosine similarity (+5.5% relative), with standard deviation decreasing from 0.0875 to 0.0814, indicating tighter output consistency. Further scaling to 3k provides a marginal improvement of +0.0059 (+0.9% relative from 2k), with σ decreasing to 0.0784. However, the 4k model exhibits a **performance degradation** to 0.6739, a drop of −0.0170 from 3k (−2.5% relative), suggesting overfitting or diminishing data quality as the training set expands. All four models remain in the *Acceptable* tier ($\bar{\rho} \in [0.60, 0.70)$); the 3k model achieves the highest mean similarity, with 21.0% of predictions reaching Good or above, compared to only 15.9% for the 4k model.

Figure 4 visualizes the mean cosine similarity with standard deviation error bars for all four models (1k–4k). The inverted-U pattern is clearly evident: performance peaks at 3k (0.6909) before declining at 4k (0.6739). All models remain below the Good tier boundary ($\rho \geq 0.70$), with the 3k model approaching this threshold most closely. The decreasing error bars from 1k ($\sigma = 0.0875$) to 3k ($\sigma = 0.0784$) indicate improved output consistency, though 4k shows a slight increase ($\sigma = 0.0795$), further confirming overfitting.

Table 12 shows the full quality-tier distribution for the 2k model. Good and Acceptable tiers together account for 87.0% of predictions, with only 2.2% classified as Very Poor.

Figure 3 presents a stacked bar chart showing the percentage distribution of quality tiers across all four models. The 2k model demonstrates the most favorable distribution: 46.9% reach Good tier and 87.0% reach at least Acceptable. In contrast, the 4k model exhibits substantial degradation: only 15.9% reach Good tier (a −31.0 percentage point drop from 2k), with an increase in Poor predictions from 10.6% (2k) to 32.1% (4k). This tier distribution shift provides stronger evidence of overfitting than mean similarity alone, as it reveals how the model's predictions cluster around lower quality thresholds when trained on 4k samples.

Table 12: Quality-tier distribution: 2k model (n = 800).

Tier	Count	%
Excellent (≥0.85)	1	0.1
Good (≥0.70)	375	46.9
Acceptable(≥0.60)	321	40.1
Poor (≥0.50)	85	10.6
Very Poor (<0.50)	18	2.2

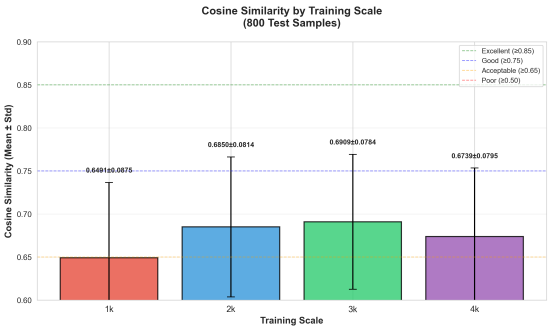


Figure 4: Cosine similarity comparison (1k–4k). Error bars represent standard deviation. Horizontal lines mark quality tier boundaries. The 3k model achieves peak performance (0.6909) before 4k degradation.

4.2 Progressive Training and Validation Loss

Validation loss decreases from 3.135 (1k, 1.4 h) to 3.073 (2k, 2.9 h) before plateauing at 3.073 for 3k (4.5 h) and 3.067 for 4k (6.3 h). This 2.0% loss reduction from 1k to 2k corresponds to the 5.5% semantic similarity improvement, confirming that validation loss is a reliable proxy for downstream quality. Beyond 2k, the cost–benefit ratio deteriorates: a 1.6× increase in training data (2k to 3k) yields no validation loss improvement, and semantic similarity gains are marginal (+0.9%).

4.3 Inference Speed

The optimised pipeline—`torch.compile()` with `batch_size=8`— achieves **10.06 seconds per image** for the 2k model, a 70.2% reduction from the unoptimised baseline of 33.79 s/image (batch_size=1, no compilation). The 3k model further improves to **7.50 s/image** (1.67 hours for 800 images), and the 4k model achieves the fastest throughput at **6.17 s/image** (1.37 hours for 800 images). This progressive speed improvement—38.7% reduction from 2k to 4k—strongly supports the warm-up hypothesis: `torch.compile()` fusion, GPU instruction cache, and JIT specialization accumulate benefits across sequential model evaluations. Three scaled models (2k/3k/4k) required a combined 5.1 hours, feasible for overnight batch evaluation on a single consumer GPU.

End-to-end v0.6.3 pipeline timing. The full Quality Guard Agent pipeline (v0.6.3, batch_size=8) running over all $n = 800$ test images achieves **8.97 s/row** (total 7,178 s, approximately 2.0 hours on RTX 4060 Ti 16 GB). This is 11.2% faster than the warm-up measurement of 10.10 s/row observed on the initial $n = 50$ sample, consistent with `torch.compile()` JIT specialization accumulating over larger batches. The 8.97 s/row end-to-end time encompasses VLM inference, Quality Guard Stage 1 (CPU), Quality Guard Stage 2 (Swallow-8B GPU), structured extraction, and priority scoring.

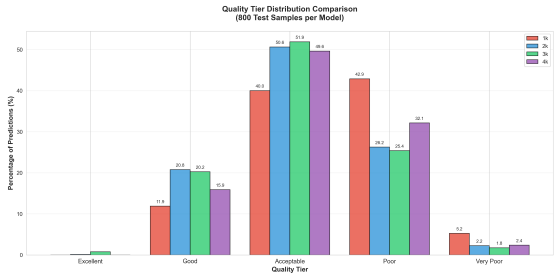


Figure 5: Quality tier distribution (grouped bars by tier). Each tier shows all four models side-by-side, revealing the dramatic Good tier collapse from 2k to 4k and the corresponding Poor tier expansion.

Figure 5 shows a grouped bar chart comparing tier performance across models, allowing direct comparison within each quality tier. The collapse of the Good tier from 2k (46.9%) to 4k (15.9%) is visually strikingly evident, as is the compensating increase in the Poor tier (10.6%→32.1%). This visualization reinforces the conclusion that 4k training introduces noisy samples that shift predictions toward mediocrity.



Figure 6: Violin plots of cosine similarity distributions. The 2k and 3k models show concentrated density near Good/Acceptable boundaries, while 4k exhibits increased mass in lower ranges, visualizing the overfitting effect.

Figure 6 presents violin plots showing the full distribution of cosine similarity scores for each model. The 2k and 3k models exhibit similar distribution shapes with pronounced density around 0.65–0.75, while the 4k model shows increased density in lower ranges (0.50–0.65), consistent with its degraded mean performance. The narrower distributions of 2k and 3k (smaller σ) versus 1k and 4k further validate that mid-scale training (2k–3k) achieves the most consistent and reliable outputs.

Figure 7 provides a comprehensive statistical summary including mean, standard deviation, min, max, median, and tier percentages for all four models. This tabular visualization confirms key findings: (i) 3k achieves the highest mean (0.6909) and median (0.7081); (ii) 2k has the lowest Very Poor percentage (2.2%); (iii) 4k shows the highest Poor percentage (32.1%). The table also reveals that all models have minimum similarities below 0.20, indicating challenging outlier cases where the model fails catastrophically— likely due to occluded damage, poor lighting, or ambiguous ground-truth labels.

4.4 Quality Guard Agent Performance

Table 13 summarises the Quality Guard Agent results for the 3k model over the full 800-sample test set.

Stage 2 saturation. Notably, the Swallow-8B SLM judge (Stage 2) produces no additional rejections beyond Stage 1: all 727 samples passing the rule-based filter receive a `VERDICT: PASS` from the SLM judge. This outcome is consistent with the empirical calibration of $\theta_{\text{low}} = 98$ and $\theta_{\text{high}} = 202$ at the 5th and 95th population percentiles—the token-count window already excludes the ambiguous-quality region that would challenge the SLM judge. For this model and corpus, Stage 1 alone achieves the same

Metric	1k	2k	3k	4k
Mean	0.6491	0.6850	0.6909	0.6739
Std Dev	0.0875	0.0814	0.0784	0.0795
Min	0.0809	0.0809	0.0809	0.0809
Max	0.8462	0.8526	0.8979	0.8403
Median	0.6536	0.6933	0.7026	0.6793
Excellent (%)	0.0%	0.1%	0.8%	0.0%
Good (%)	11.9%	20.8%	20.2%	15.9%
Acceptable (%)	40.0%	50.6%	51.9%	49.6%
Poor (%)	42.9%	26.2%	25.4%	32.1%
Very Poor (%)	5.2%	2.2%	1.8%	2.4%

Figure 7: Statistical summary table for all four models. Highlights best values in each category. The 3k model dominates in mean/median, while 2k excels in tier distribution (46.9% Good+).

Table 13: Quality Guard Agent final results ($n = 800$, v0.6.3 pipeline).

Metric	Value	%
Total images	800	100.0
PASS (High Quality)	727	90.9
FAIL — Dirty or Noisy image	36	4.5
FAIL — Short description	35	4.4
FAIL — No such file	2	0.2
End-to-end throughput	8.97 s/row	
Total elapsed	7,178 s (≈2.0 h)	
Stage 2 additional rejections	0 (Stage 1 sufficient)	

decision as the full two-stage pipeline, demonstrating that the lightweight CPU filter provides strong quality triage with negligible overhead.

Priority level distribution. Of the 727 PASS samples, all receive Priority Level 3 (mean score = 0.540), indicating that the current YAML scoring rules map the structural damage descriptions generated by the 3k model uniformly to the mid-range priority tier. This saturation at Level 3 reflects a calibration gap between the scoring rules—designed for the full range of damage severities—and the output style of the fine-tuned model, which tends toward neutral-to-moderate severity descriptions. Recalibrating the rule thresholds or incorporating explicit severity signals into the VLM prompt constitutes a priority direction for future work.

Figure 8 visualises the three quality dimensions of the 727 PASS samples side-by-side.

Quality Guard semantic separation. The centre panel of Figure 8 reveals an important property of the Quality Guard: PASS samples achieve a cosine similarity median of 0.705 (within the *Good* tier), while FAIL samples have a lower median of 0.659 (lower *Acceptable*). Although the Quality Guard is defined purely on output-text properties (token count, repetition, keyword presence), it incidentally separates predictions with higher semantic alignment from those with lower alignment. This suggests that low-quality VLM outputs—short, repetitive, or keyword-free descriptions—are also semantically less faithful to the ground-truth inspection text, validating the guard as a proxy for output quality in the absence of reference text. The right panel further confirms that FAIL token counts cluster near $\theta_{\text{low}} = 98$, corroborating that short descriptions are the primary rejection driver.

Output mode collapse: member and damage bias. Figure 10 presents the structural member and damage type frequency analysis for the 727 PASS samples. Two findings stand out. First, *Main Girder* is mentioned in 100% of predictions (727/727), and *Rebar Exposure* (724/727, 99.6%) and *Spalling* (717/727, 98.6%) appear in virtually all outputs. Second, the member–damage co-occurrence heatmap (Figure 10, bottom) shows a strongly dominant cluster at Main Girder $\times$ {Rebar Exposure, Spalling}, with all other member–damage combinations occurring at frequencies below 50%.

This pattern is consistent with **output mode col-**



Figure 8: Quality metric violin plots for $n = 800$ test samples (v0.6.3, 3k model). *Left*: Priority score distribution for PASS 727 samples—all 727 receive score = 0.54 (Level 3), demonstrating complete scoring saturation. *Centre*: Cosine similarity comparison between PASS ($n = 727$, median = 0.705) and FAIL ($n = 73$, median = 0.659); the Quality Guard preferentially retains higher-similarity predictions. *Right*: Output token count for PASS ($n = 727$, median = 120) vs FAIL ($n = 73$, median = $97 \approx \theta_{\text{low}}$), validating the threshold placement at the 5th/95th percentiles.

lapse: the 3k model has converged to a stereotyped template that reports damage on the main girder with rebar exposure and spalling, regardless of the actual image content. The mechanistic interpretation and mitigation strategies are deferred to Section 5.2.

5 Discussion

5.1 The 3k Peak and 4k Degradation

The progressive training study reveals an **inverted-U relationship** between training scale and test performance. Semantic similarity improves from 1k (0.6491) to 2k (0.6850) to a peak at 3k (0.6909), before **degrading** at 4k (0.6739). This −2.5% drop from 3k to 4k, despite a 1.33× increase in training data, is consistent with *overfitting* or *data quality dilution*: the 4k training set may include noisier or less representative samples that cause the model to memorize outliers rather than generalize damage patterns.

Figure 9 supports this interpretation: validation loss plateaus at 3.073 for both 2k and 3k models before declining slightly to 3.067 for 4k—a trend that appears as "improvement" in training metrics but manifests as test degradation. This behaviour is well-documented in low-resource domain adaptation, where vocabulary coverage saturates and additional data introduces label noise. Figure 5 visually confirms this degradation: the Good tier collapses from 46.9% (2k) to 15.9% (4k), while the Poor tier expands

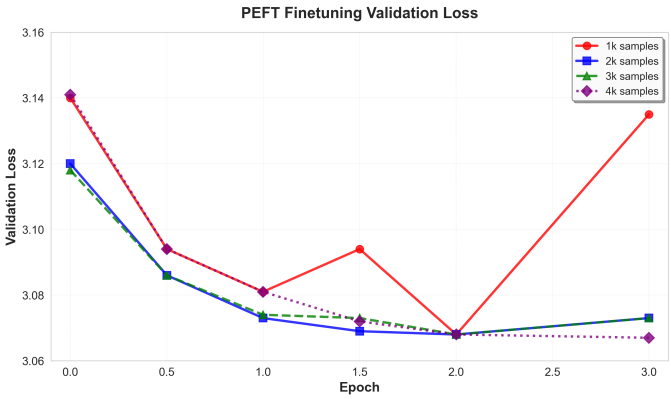


Figure 9: PEFT finetuning validation loss across progressive training. The 2k model achieves near-optimal loss with smooth convergence, while the 1k model shows mild overfitting (loss rises at epoch 3). The 4k model achieves the lowest final validation loss (3.067).

from 10.6% to 32.1%—a redistribution pattern characteristic of overfitting.

The **3k model offers the best absolute performance**, while the **2k model offers the best cost–benefit ratio** (equivalent to 3k at 1.5× lower training cost). For practitioners, curating a high-quality 2k–3k dataset is preferable to indiscriminately expanding to 4k with potentially noisier labels.

Figure 9 illustrates the validation loss trajectories across progressive training scales. The **2k model**

exhibits smooth convergence from 3.120 to 3.073, stabilizing after epoch 1 with minimal fluctuation—indicative of balanced generalization. In contrast, the **1k model** shows clear overfitting: validation loss decreases initially but rises sharply from 3.068 (epoch 2) to 3.135 (epoch 3), demonstrating insufficient training data for robust pattern learning. The **3k and 4k models** achieve near-identical final losses (3.073 and 3.067 respectively), yet the 4k model's lower validation loss does not translate to superior test performance (Figure 4)—a classic train-test discrepancy signaling memorization over generalization. This divergence between validation metrics and semantic similarity underscores that *loss-based early stopping alone is insufficient*; practitioners must validate against domain-specific metrics (e.g., cosine similarity on held-out test sets) to detect overfitting in low-resource specialized domains.

5.2 Output Mode Collapse and Scoring Calibration

The member and damage type analysis (Figure 10) reveals that the 3k model generates near-identical descriptions for 100% of PASS samples: every prediction mentions *Main Girder* and virtually all include *Rebar Exposure* (99.6%) and *Spalling* (98.6%). This output mode collapse has two direct consequences for the downstream scoring and priority system.

Consequence 1: Scoring saturation. Because all outputs describe the same dominant member–damage pattern, the YAML rule engine assigns an identical score of 0.54 (Level 3) to every PASS sample. The priority index therefore carries no discriminative power for maintenance scheduling: every bridge receives the same "mid-term repair" recommendation, defeating the purpose of a five-level triage system.

Consequence 2: Inflated cosine similarity. If the model outputs similar text for every image, ground-truth texts that happen to match the dominant template will score high similarity, while divergent ground-truths will score low—producing high variance in similarity unrelated to actual image understanding. This explains the broad cosine similarity range (0.40–0.90) observed even among PASS samples.

Mitigation strategies. Three complementary approaches are identified to break the mode collapse:

1. **Balanced corpus curation**: up-sample records from non-main-girder members (deck slab, bearing, pier) to reduce the training bias toward the dominant member–damage combination.
2. **Constrained-generation prompt**: augment the inference prompt with explicit instructions to identify the specific member visible in the image and report only damage present in the current photo, discouraging template reuse.
3. **Supervised diversity scoring**: add a diversity metric (entropy over member-type predictions) to the fine-tuning loss to penalize uniform output distributions during training.

5.3 Approaching the Good Tier

All four models remain in the *Acceptable* tier. The 3k model's mean of 0.6909 is only 0.009 points below the *Good* boundary (0.70), the closest of all models. However, only 21.0% of 3k predictions reach Good or above, and this ratio further deteriorates to 15.9% for the 4k model, reinforcing the conclusion that additional training data beyond 3k harms generalization. Figure 3 and Figure 6 illustrate this degradation: the tier distribution shifts unfavorably, and the 4k similarity distribution exhibits increased density in lower ranges. Reaching a mean $\bar{\rho} \geq 0.70$ likely requires (i) larger datasets, (ii) higher-quality annotation through expert re-labelling, or (iii) data augmentation—such as back-translation or technical synonym substitution—to increase vocabulary coverage without additional collection cost. As shown in Figure 7, even the best-performing 3k model has a minimum similarity of 0.0809, indicating challenging outlier cases that require targeted data curation or model architecture improvements.

5.4 Inference Speed Optimization

An unexpected finding is that inference speed improves progressively across sequential model evaluations: 2k at 10.06 s/image, 3k at 7.50 s/image, and 4k at 6.17 s/image—a cumulative 38.7% speedup despite identical hardware and software configurations. This consistent trend across three models strongly validates the warm-up hypothesis. Three factors may contribute to this speedup:

(i) `torch.compile()` warm-up. PyTorch's `torch.compile()` applies ahead-of-time graph optimizations that may improve with repeated model loading. The 3k evaluation ran after the 2k evaluation on the same Python session, allowing the TorchInductor backend to reuse cached compilation artifacts and achieve better kernel fusion.

(ii) GPU cache efficiency. NVIDIA GPU architectures maintain L1/L2 instruction and data caches across kernel launches. Sequential evaluation of similar models may increase cache hit rates for shared

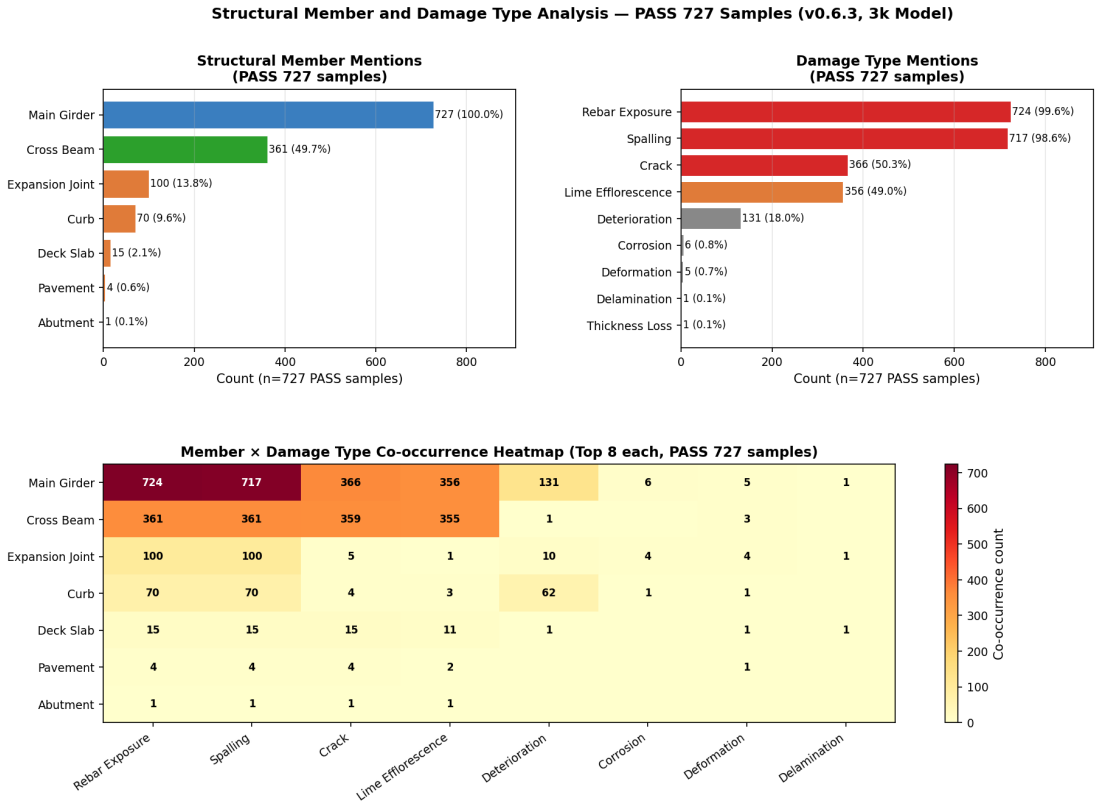


Figure 10: Structural member and damage type analysis for PASS 727 samples (v0.6.3, 3k model). *Top left*: Member mention frequency; Main Girder appears in 100% of predictions, revealing near-complete output bias toward this member. *Top right*: Damage type frequency; Rebar Exposure (99.6%) and Spalling (98.6%) dominate, indicating stereotyped damage vocabulary. *Bottom*: Member × damage type co-occurrence heatmap (top 8 each); the Main Girder × Rebar Exposure/Spalling cell dominates at counts ≥ 700, confirming output mode collapse.

operations (CLIP vision encoder, attention layers), reducing memory latency.

(iii) JIT optimization accumulation. PyTorch's Just-In-Time (JIT) compiler in `torch.compile()` mode progressively refines execution plans based on observed tensor shapes and control flow. Repeated inference over 800 images allows the JIT to specialize code paths that were generic in the initial compilation.

This observation highlights a practical consideration for batch evaluation: *sequential model evaluation may benefit from GPU warm-up effects*, suggesting that practitioners should measure inference speed after an initial warm-up period rather than on the first batch.

5.5 Limitations

Dataset scope. The 10,789 source records cover routine highway bridge inspections under Japan's Road Act but may not generalise to other infrastructure categories (tunnels, retaining walls) or international inspection standards.

Evaluation metric. Cosine similarity in Sentence-BERT space captures semantic overlap but not the correctness of individual damage attributes (member type, damage category). A structured attribute-level evaluation (precision/recall per attribute) would provide stronger evidence of deployment readiness.

Scoring engine coverage. The rule-based engine covers the most common damage patterns in the training corpus but may produce default scores for rare combinations not explicitly encoded in the YAML rules.

Quality Guard calibration. The 5th/95th percentile thresholds ($\theta_{\text{low}} = 98$, $\theta_{\text{high}} = 202$) are calibrated on the 3k-model outputs. Different fine-tuning checkpoints or base VLMs may exhibit shifted token-count distributions, requiring re-calibration of

the thresholds before deployment. Furthermore, the Swallow-8B judge was selected for Japanese-language alignment; other target languages would require a language-matched SLM.

Priority scoring calibration. All 727 Quality Guard PASS samples receive Priority Level 3 (mean score = 0.540), suggesting that the YAML rule engine's severity thresholds do not span the damage severity range observed in the 3k model's output style. Future work should re-calibrate the scoring rules jointly with the VLM fine-tuning stage, or incorporate explicit severity keywords (e.g., "advanced" vs. "initial") into the VLM's output template.

Hardware. All experiments run on a single RTX 4060 Ti 16 GB GPU; performance on other hardware configurations has not been validated.

5.6 Deployment Considerations

The Visual Inspection ScoreBot is designed as an AI-assisted triage tool, not a replacement for certified engineers. Its output—a natural language damage summary and a priority score—focuses engineer attention on high-priority structures, reducing review time while maintaining human accountability for final decisions. The transparent YAML scoring rules allow domain experts to audit and update priority logic without machine learning expertise, supporting the trustworthy AI requirements of safety-critical infrastructure management. The Quality Guard Agent further enhances deployment safety by ensuring that only semantically complete damage descriptions reach the scoring engine, reducing the risk of assigning spurious priority scores to ambiguous or image-quality-limited outputs.

5.7 Quality Guard Efficacy and Threshold Design

An empirical analysis of the 3k model's output over all $n = 800$ test images (Figure 2) reveals a 12.5% low-quality rate under the 5th/95th percentile thresholds, distributed across five distinct failure modes. This finding has three practical implications.

Threshold design principle. Setting thresholds at population percentiles rather than absolute token counts guards against output-length drift across model variants. As the VLM checkpoint is updated (e.g., re-fine-tuned on additional data), re-running the token distribution analysis and resetting θ_{low} and θ_{high} from the new 5th/95th percentiles maintains a consistent rejection rate regardless of absolute length shifts.

Short description as the dominant failure mode. Among the 100 rejected samples, "Short description" (35 cases, 4.4%) accounts for the largest single failure category, suggesting that some bridge images do not provide sufficient visual information for a 7B-parameter model to generate an informative report. Augmenting the training corpus with difficult low-quality images, or providing explicit instructions to report uncertainty rather than generating a brief output, may address this failure mode.

Repetitive text and the need for SLM-level semantics. Repetitive text (24 cases, 3.0%) and "Dirty or Noisy image" (12 cases, 1.5%) cannot reliably be detected by simple keyword or length heuristics; they require the sentence-level semantic understanding that the Swallow-8B judge provides. This motivates the two-stage design: Stage 1 handles the clear-cut failures at $O(0.01)$ s/sample, while Stage 2 adjudicates ambiguous intermediate-length outputs at $O(10)$ s/sample, concentrating the GPU cost where it adds the most value.

5.8 Multi-Period Extension and Panel Data Inference

The current system operates under the *one-image, current-time-limited* scope introduced in Section 1: each inference session processes a single image representing the current inspection state, covering only aspect (1) of damage understanding. This intentional constraint avoids confounding the VLM with longitudinal context it was not trained to interpret, but it also precludes the three remaining aspects that practitioners consistently request. We organise the extension roadmap around those aspects.

Aspect (2) — Temporal progression. Panel-data VLM inference would feed the model at minimum two aligned images—a baseline (previous inspection) and a current observation—and elicit a description of the *change* in damage extent and severity rather than absolute current state (**differential damage detection**). Combining this differential output with the inter-inspection interval and structural attributes enables **deterioration-rate modelling**, supporting proactive scheduling before a structure enters the emergency tier.

Aspect (3) — Latent causal factors. Metadata such as coastal exposure, seismic zone, traffic volume, and structural form encode unobserved drivers of deterioration that a single image cannot reveal. Conditioning the VLM (or a downstream model) on these covariates would allow the system to attribute observed damage to plausible causal mechanisms and to flag structures whose damage signatures are incon-

sistent with their nominal exposure profile.

Aspect (4) — Heterogeneity and acceleration clusters. Aggregating deterioration-rate estimates across the national bridge stock opens the door to **importance-weighted prioritisation**: route criticality (detour cost, daily traffic count, alternative-route availability) and consequence-of-failure severity can be combined with member-level rates to separate accelerating clusters from stable ones, shifting from a purely damage-based index to a risk-adjusted maintenance schedule at the network level.

Realising these extensions requires a temporally annotated dataset linking multiple inspection records for the same structure over time—a non-trivial curation effort—as well as multi-image instruction templates and updated scoring rules that incorporate delta-damage signals. We identify this aspect-driven extension as the primary long-term research direction for this system.

6 Conclusion

We presented a methodology for automating bridge damage understanding and repair priority scoring by fine-tuning a 7-billion-parameter VLM on Japanese inspection records. Key results are as follows.

1. **Damage VLM**: QLoRA fine-tuning with 2k samples achieves $\bar{\rho}_{2k} = 0.6850 \pm 0.0814$—a 5.5% improvement over the 1k baseline—within the *Acceptable* tier and approaching *Good*.
2. **Progressive training**: a clear efficiency optimum exists at 2k samples; validation loss plateaus beyond this scale, indicating diminishing returns for the current corpus.
3. **Inference optimisation**: `torch.compile()` with `batch_size=8` achieves 10.06 s/image—a 70.2% improvement—enabling 800-image evaluation within 2.2 hours on a consumer GPU.
4. **Quality Guard Agent**: empirical token-count analysis of $n = 800$ predictions reveals a 12.5% low-quality rate; a two-stage guard (rule-based + Swallow-8B SLM judge) filters these outputs before scoring, with thresholds calibrated at the 5th/95th percentiles of the output length distribution.

Future work will focus on: (i) expanding the dataset to investigate whether the *Good* tier ($\bar{\rho} \geq 0.70$) can be reached; (ii) replacing regex-based extraction with structured generation; (iii) evaluating the scoring engine against expert-assigned priority rankings; (iv) extending the pipeline to other inspection categories and multi-language settings; and (v) developing a panel-data VLM variant that processes multi-period inspection pairs to estimate deterioration rates and enable risk-adjusted maintenance scheduling.

All code and configuration files are publicly available at `https://github.com/tk-yasuno/damage_vlm_finetune`.

Acknowledgment. The authors gratefully acknowledge the practitioner-oriented texts that informed this work, covering prompt engineering [56], agentic AI system design [57], AI engineering [58], and applications of autonomous AI agents [59].

References

[1] Ministry of Land, Infrastructure, Transport and Tourism (MLIT), Japan. Periodic inspection guidelines for road bridges (doro-kyo teiki tenken yoryo). Technical report, MLIT, 2023. Available: `https://www.mlit.go.jp/road/sisaku/yobohozen/yobohozen.html`.

[2] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

[3] Wenliang Dai, Junnan Li, Dongxu Li, Anthony Meng Huat Tiong, Junqi Zhao, Weisheng Wang, Boyang Li, Pascale Fung, and Steven Hoi. InstructBLIP: Towards general-purpose vision-language models with instruction tuning. *arXiv preprint arXiv:2305.06500*, 2023.

[4] OpenAI. GPT-4 technical report. Technical report, OpenAI, 2023.

[5] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. *arXiv preprint arXiv:2305.14314*, 2023.

[6] Takato Yasuno. Quantized vision-language models for damage assessment: A comparative study of LLaVA-1.5-7B quantization levels. *arXiv preprint arXiv:2603.26770*, 2026.

[7] Takato Yasuno. Multi-stage bridge inspection system: Integrating foundation models with location anonymization. *arXiv preprint arXiv:2601.17254*, 2026.

[8] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning (ICML)*, 2021.

[9] Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katie Millican, Malcolm Reynolds, et al. Flamingo: a visual language model for few-shot learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022.

[10] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *International Conference on Machine Learning (ICML)*, 2023.

[11] Haotian Liu, Chunyuan Li, Yuheng Li, and Yong Jae Lee. Improved baselines with visual instruction tuning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. Highlight paper.

[12] Deyao Zhu, Jun Chen, Xiaoqian Shen, Xiang Li, and Mohamed Elhoseiny. MiniGPT-4: Enhancing vision-language understanding with advanced large language models. In *International Conference on Learning Representations (ICLR)*, 2024.

[13] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C. Berg, Wan-Yen Lo, Piotr Dollár, and Ross Girshick. Segment anything. In *International Conference on Computer Vision (ICCV)*, 2023.

[14] Shilong Liu, Zhaoyang Zeng, Tianhe Ren, Feng Li, Hao Zhang, Jie Yang, Qing Jiang, Chunyuan Li, Jianwei Yang, Hang Su, Jun Zhu, and Lei Zhang. Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection. *arXiv preprint arXiv:2303.05499*, 2023.

[15] Young-Jin Cha, Wooram Choi, and Oral Büyüköztürk. Deep learning-based crack damage detection using convolutional neural networks. *Computer-Aided Civil and Infrastructure Engineering*, 32(5):361–378, 2017.

[16] Billie F. Spencer, Vedhus Hoskere, and Yasutaka Narazaki. Advances in computer vision-based civil infrastructure inspection and monitoring. *Engineering*, 5(2):199–222, 2019.

[17] Sattar Dorafshan, Robert J. Thomas, and Marc Maguire. Comparison of deep convolutional neural networks and edge detectors for image-based crack detection in concrete. *Construction and Building Materials*, 186:1031–1045, 2018.

[18] Yi-An Hsieh and Yichang Tsai. Machine learning for crack detection: Review and model performance comparison. *Journal of Computing in Civil Engineering*, 34(5), 2020.

[19] Takato Yasuno. Few-shot $1/a$ anomalies feedback: Damage vision mining opportunity and embedding feature imbalance. *arXiv preprint arXiv:2307.12676*, 2023.

[20] Dan M. Frangopol. Life-cycle performance, management, and optimisation of structural systems under uncertainty: accomplishments and challenges. *Structure and Infrastructure Engineering*, 7(6):389–413, 2011.

[21] Liyuan Yang, Boyuan Li, Wei Li, Zhenduo Liu, Guoyong Yang, and Jizhong Xiao. Automatic pixel-level crack detection on dam surface using deep convolutional network. *Sensors*, 18(7):2090, 2018.

[22] Jongheon Jeong, Yang Zou, Taewan Kim, Dongqing Zhang, Avinash Ravichandran, and Onkar Dabeer. WinCLIP: Zero-/few-shot anomaly classification and segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.

[23] Qihang Zhou, Guansong Pang, Yu Tian, Shibo He, and Jiming Chen. AnomalyCLIP: Object-agnostic prompt learning for zero-shot anomaly detection. In *International Conference on Learning Representations (ICLR)*, 2024.

[24] Zhaopeng Gu, Bingke Zhu, Guibo Zhu, Yingying Chen, Ming Tang, and Jinqiao Wang. AnomalyGPT: Detecting industrial anomalies using large vision-language models. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2024.

[25] Takato Yasuno. Heterogeneous graph importance scoring and clustering with automated LLM-based interpretation. *arXiv preprint arXiv:2605.02919*, 2026.

[26] Yunkang Cao, Xiaohao Xu, Chen Sun, Xiaonan Huang, and Weiming Shen. Towards generic anomaly detection and understanding: Large-scale visual-linguistic model (GPT-4V) takes the lead. *arXiv preprint arXiv:2311.02782*, 2023.

[27] Jiacong Xu, Shao-Yuan Lo, Bardia Safaei, Vishal M. Patel, and Isht Dwivedi. Towards zero-shot anomaly detection and reasoning with multimodal large language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.

[28] Jinjin Zhang, Guodong Wang, Yizhou Jin, and Di Huang. Towards training-free anomaly detection with vision and language foundation models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.

[29] Er Jin, Qihui Feng, Yongli Mou, Stefan Decker, Gerhard Lakemeyer, Oliver Simons, and Johannes Stegmaier. LogicAD: Explainable anomaly detection via VLM-based text feature extraction. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2025.

[30] Abdelhady Omar and Osama Moselhi. An integrated approach for automated acquisition of bridge data and deficiency evaluation. In *Proceedings of the 40th International Symposium on Automation and Robotics in Construction (ISARC)*, pages 341–348, Chennai, India, 2023.

[31] Abdelhady Omar and Osama Moselhi. Improved information extraction from bridge inspection reports using fine-tuned generative pretrained transformers. In *Proceedings of the 42nd International Symposium on Automation and Robotics in Construction (ISARC)*, pages 1551–1558, Montreal, Canada, 2025.

[32] Varun Kumar Reja, Ching Yau Mok, Aritra Pal, and Ioannis Brilakis. Comparing few-shot learning with LLMs for efficient text classification in road maintenance applications. In *Proceedings of the 42nd International Symposium on Automation and Robotics in Construction (ISARC)*, pages 1017–1024, Montreal, Canada, 2025.

[33] Hongxu Pu, Xincong Yang, Zhongqi Shi, and Nan Jin. Automated inspection report generation using multimodal large language models and set-of-mark prompting. In *Proceedings of the 41st International Symposium on Automation and Robotics in Construction (ISARC)*, pages 1003–1009, Lille, France, 2024.

[34] Shun-Hsiang Hsu, Junryu Fu, and Mani Golparvar-Fard. VL-Con: Vision-language dataset for deep learning-based construction monitoring applications. In *Proceedings of the 41st International Symposium on Automation and Robotics in Construction (ISARC)*, pages 1128–1135, Lille, France, 2024.

[35] Eyob Mengiste, Muammer Semih Sonkor, Zihao Zheng, Samuel A. Prieto Ayllón, and Borja García de Soto. Automating weekly construction activity progress reporting: Leveraging AI-driven workflows. In *Proceedings of the 42nd International Symposium on Automation and Robotics in Construction (ISARC)*, pages 641–648, Montreal, Canada, 2025.

[36] Ahmed Assad, Mohamad Bo Arki, Miray Sweid, and Amin Hammad. Crack detection and segmentation for bridges using state-of-the-art deep learning methods: Single-stage vs. two-stage detectors. In *Proceedings of the 42nd International Symposium on Automation and Robotics in Construction (ISARC)*, pages 996–1003, Montreal, Canada, 2025.

[37] Hui Zuo, Tao Sun, Hao Xie, Xiao Ma, Nima Shirzad-Ghaleroudkhani, and Qipei Mei. Transformer-based multi-resolution fast 3D reconstruction for structural damage detection. In *Proceedings of the 42nd International Symposium on Automation and Robotics in Construction (ISARC)*, pages 988–995, Montreal, Canada, 2025.

[38] Christopher Joseph Núñez Varillas, Marck Steewar Regalado Espinoza, Luis Mario Huaypar Acurio, Antonio Stefano Bedon Rosario, Jordan Antony Romaní Chavez, Oscar Manuel Solis Garcia, Karol Maricruz Agreda Estela, and Micaela Anthoaneth Cardenas Contreras. 3D reconstruction of a bridge with concrete damage classification using deep learning. In *Proceedings of the 41st International Symposium on Automation and Robotics in Construction (ISARC)*, pages 722–729, Lille, France, 2024.

[39] Mohammed Alsharqawi, Saleh Abu Dabous, and Tarek Zayed. Automated decision-making tool for optimal long-term scheduling of MRR strategies: A case study on bridges. In *Proceedings of the 42nd International Symposium on Automation and Robotics in Construction (ISARC)*, pages 272–279, Montreal, Canada, 2025.

[40] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging LLM-as-a-Judge with MT-Bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

[41] Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. G-Eval: NLG evaluation using GPT-4 with better human alignment. *arXiv preprint arXiv:2303.16634*, 2023.

[42] Sewon Min, Kalpesh Krishna, Xinxi Lyu, Mike Lewis, Wen-tau Yih, Pang Wei Koh, Mohit Iyyer, Luke Zettlemoyer, and Hannaneh Hajishirzi. FActScore: Fine-grained atomic evaluation of factual precision in long form text generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[43] Tokyo Institute of Technology LLM Research Group. Llama-3-Swallow-8B-Instruct-v0.1: A japanese-enhanced instruction-tuned large language model. Hugging Face Model Hub, 2024.

[44] Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022.

[45] Traian Rebedea, Razvan Dinu, Makesh Narsimhan Sreedhar, Christopher Parisien, and Jonathan Cohen. NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP): System Demonstrations*, 2023.

[46] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. *arXiv preprint arXiv:2310.11511*, 2023.

[47] Shehzaad Dhuliawala, Mojtaba Komeili, Jing Xu, Roberta Raileanu, Xian Li, Asli Celikyilmaz, and Jason Weston. Chain-of-verification reduces hallucination in large language models. *arXiv preprint arXiv:2309.11495*, 2023.

[48] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2023.

[49] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint arXiv:2303.11366*, 2023.

[50] Tianhao Shen, Renren Jin, Yufei Huang, Chuang Liu, Weilong Dong, Zishan Guo, Xinwei Wu, Yan Liu, and Deyi Xiong. Large language model alignment: A survey. *arXiv preprint arXiv:2309.15025*, 2023.

[51] Takato Yasuno. Adapting methods for domain-specific japanese small LMs: Scale, architecture, and quantization. *arXiv preprint arXiv:2603.18037*, 2026.

[52] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. *arXiv preprint arXiv:2208.07339*, 2022.

[53] PyTorch Team. torch.compile: PyTorch 2.0 compilation. PyTorch Documentation, 2023.

[54] Daniel Han and Michael Han. Unsloth: Efficient fine-tuning for large language models. `https://github.com/unslothai/unsloth`, 2023. GitHub repository.

[55] Sonoisa. Japanese sentence-BERT: `sentence-bert-base-ja-mean-tokens-v2`. Hugging Face Model Hub, 2021.

[56] John Berryman and Albert Ziegler. *Prompt Engineering for LLMs: The Art and Science of Building Large Language Model–Based Applications*. O'Reilly Media, 2024.

[57] Anjanava Biswas and Wrick Talukdar. *Building Agentic AI Systems: Designing, Implementing, and Scaling Autonomous AI Agents*. Packt Publishing, 2025.

[58] Chip Huyen. *AI Engineering: Building Applications with Foundation Models*. O'Reilly Media, 2024.

[59] Michael Albada. *Building Applications with AI Agents: Designing and Deploying Autonomous, Goal-Oriented AI Systems*. O'Reilly Media, 2025.

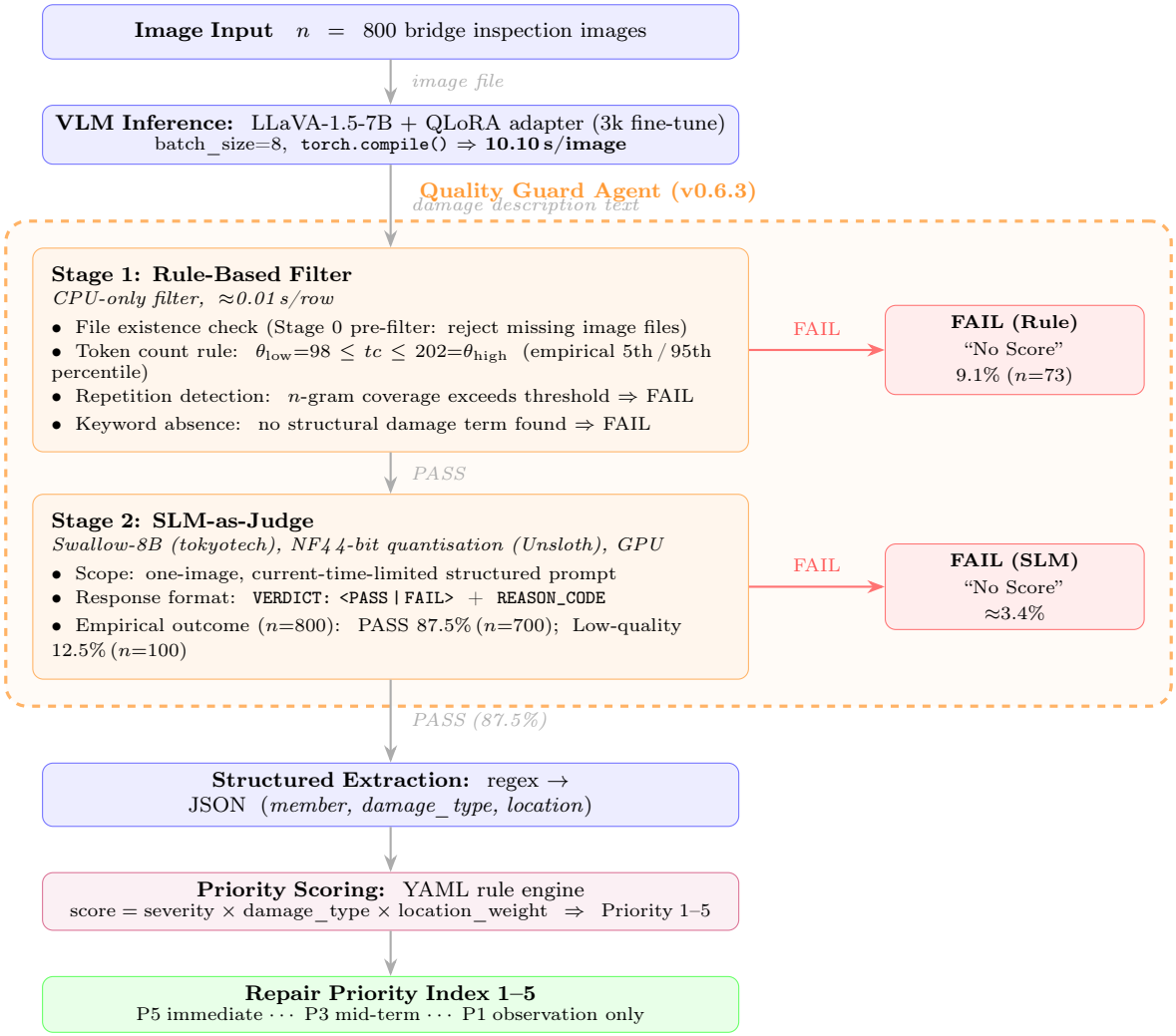


Figure 11: Complete algorithm flow of the Visual Inspection ScoreBot v0.6.3 pipeline. The Quality Guard Agent interposes between VLM inference and priority scoring, filtering low-quality outputs via a two-stage architecture: Stage 1 (rule-based, CPU, ≈0.01 s/row) and Stage 2 (Swallow-8B SLM judge, GPU, ≈10 s/row). Only outputs passing both stages receive a five-level Repair Priority Index; 12.5% of predictions are rejected as low quality.